

Pandas is a Python package providing fast, flexible, and expressive data structures designed to make working with structured (tabular, multidimensional, potentially heterogeneous) and time series data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, real world data analysis in Python.

Pandas is designed to make it easier to work with structured data. Most of the analyses you might perform will likely involve using tabular data, e.g., from .csv files or relational databases (e.g., SQL). The DataFrame object in pandas is "a two-dimensional tabular, column-oriented data structure with both row and column labels."

Here are just a few of the things that pandas does well:

- Robust IO tools for loading data from flat files (CSV and delimited), Excel files, databases, and saving / loading data from the ultrafast HDF5 format
- Easy handling of missing data (represented as NaN) in floating point as well as non-floating point data
- Size mutability: columns can be inserted and deleted from DataFrame and higher dimensional objects
- Automatic and explicit data alignment: objects can be explicitly aligned to a set of labels, or the user can simply ignore the labels and let Series, DataFrame, etc. automatically align the data for you in computations
- Powerful, flexible group by functionality to perform split-apply-combine operations on data sets, for both aggregating and transforming data
- Make it easy to convert ragged, differently-indexed data in other Python and NumPy data structures into DataFrame objects
- Intelligent label-based slicing, fancy indexing, and subsetting of large data sets
- Intuitive merging and joining data sets
- Flexible reshaping and pivoting of data sets
- Hierarchical labeling of axes (possible to have multiple labels per tick)
- Time series-specific functionality: date range generation and frequency conversion, moving window statistics, moving window linear regressions, date shifting and lagging, etc.

✓ If you're curious:

The pandas name itself is derived from panel data, an econometrics term for multidimensional structured data sets, and Python data analysis itself. After getting introduced, you can consult the full pandas documentation.

<https://pandas.pydata.org/pandas-docs/stable/?v=20190919220335>

Pandas Cheatsheet: https://assets.datacamp.com/blog_assets/PandasPythonForDataScience.pdf

```
#Import package
import pandas as pd
import numpy as np
```

```
# Version of pandas
pd.__version__
```



What are Pandas Dataframes?

- In the chapters introducing Python lists and numpy arrays, you learn that both of these data structures can store collections of values, instead of just single values. You also learned that while Python lists are flexible and can store data items of various types (e.g. integers, floats, text strings), numpy arrays require all data elements to be of the same type. Because of this requirement, numpy arrays can provide more functionality for running calculations such as element-by-element arithmetic operations (e.g. multiplication of each element in the numpy array by the same value) that Python lists do not support.
- You may now be noticing that each data structure provides different functionality that can be useful in different workflows.
- In this chapter, you will learn about Pandas dataframes, a data structure in Python that provides the ability to work with tabular data. Pandas dataframes are composed of rows and columns that can have header names, and the columns in pandas dataframes can be different types (e.g. the first column containing integers and the second column containing text strings). Each value in pandas dataframe is referred to as a cell that has a specific row index and column index within the tabular structure.

✓ There are two central concepts which you should know about:

- **Series** : A series is essentially a column
- **Data Frame** : A DataFrame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns. Pandas DataFrame consists of three principal components, the data, rows, and columns.

Series		Series		DataFrame		
	Apples		Oranges		Apples	Oranges
0	3	0	0	0	3	0
1	2	1	3	1	2	3
2	0	2	7	2	0	7
3	1	3	2	3	1	2

✓ Distinguishing Characteristics of Pandas DataFrame

These characteristics (i.e. tabular format with rows and columns that can have headers) make pandas dataframes very versatile for not only storing different types, but for maintaining the relationships between cells across the same row and/or column.

Recall that in the chapter on numpy arrays, you could not easily connect the values across two numpy arrays, such as those for precip and months. Using a pandas dataframe, the relationship between the value January in the months column and the value 0.70 in the precip column is maintained.

- These two values (Apples and 3) are considered part of the same record, representing the same observation in the pandas dataframe. In addition, pandas dataframes have other unique characteristics that differentiate them from other data structures:

- Each column in a pandas dataframe can have a label name (i.e. header name such as Oranges) and can contain a different type of data from its neighboring columns.
- By default, each row has an index within a range of values beginning at [0]. However, the row index in pandas dataframes can also be set as labels (e.g. a location name, date).
- All cells in a pandas dataframe have both a row index and a column index (i.e. two-dimensional table structure), even if there is only one cell (i.e. value) in the pandas dataframe.
- In addition to selecting cells through location-based indexing (e.g. cell at row 1, column 1), you can also query for data within pandas dataframes based on specific values (e.g. querying for specific text strings or numeric values).
- Because of the tabular structure, you can work with cells in pandas dataframes:
 - across an entire row
 - across an entire column (or series, a one-dimensional array in pandas)
 - by selecting cells based on location or specific values
- Due to its inherent tabular structure, pandas dataframes also allow for cells to have null values (i.e. no data value such as blank space, NaN, -999, etc).

✓ Let us go ahead and create our first Pandas Series

```
# Our first series
number_series = pd.Series([2, 3, 5, 6, 8])
print(number_series)
```

```
↩ 0    2
   1    3
   2    5
   3    6
   4    8
dtype: int64
```

```
# Same series but with different indices
number_series = pd.Series([2, 3, 5, 6, 8],index=['a', 'b', 'd','e','f'],name='Number')
print(number_series)
```

```
↩ a    2
   b    3
   d    5
   e    6
   f    8
Name: Number, dtype: int64
```

```
print(number_series[0])
print(number_series['a'])
```

```
↩ 2
2
/tmp/ipython-input-67-1067065090.py:1: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version,
print(number_series[0])
```

✓ Let us go ahead and create our first Pandas DataFrame

✓ Using a 2-D NumPy array

```
# Creating our first DataFrame
some_2D_array = np.random.randint(4, 6,(3,4))
first_dataframe = pd.DataFrame(some_2D_array , columns=['Akshay','Devashish','Divy','Monika'])
first_dataframe
```



	Akshay	Devashish	Divy	Monika
0	5	4	5	4
1	4	5	4	5
2	4	5	5	5

```
# Lets create the same dataframe but different indices
another_dataframe = pd.DataFrame(some_2D_array, columns=['A','B','C','D'],index = ['First','Second','Third'])
another_dataframe
```



	A	B	C	D
First	5	4	5	4
Second	4	5	4	5
Third	4	5	5	5

✓ Using a conventional list of lists

```
# Initialize list of lists
list_of_lists = [['Amar', 10], ['Akbar', 15], ['Anthony', 14]]
```

```
# Create the pandas DataFrame
df = pd.DataFrame(list_of_lists, columns = ['Name', 'Age'])
```

```
# Print dataframe
df
```



	Name	Age
0	Amar	10
1	Akbar	15
2	Anthony	14

✓ Using a dictionary

```
# Store the data in a dictionary
employee_dict = {'Employee Name':['Rajeev', 'Sumit', 'Aviral'], 'Income':[200000, 140000, 90000]}
```

```
# Create DataFrame
employee_df = pd.DataFrame(employee_dict)
```

```
# Print the dataframe
employee_df
```



	Employee Name	Income
0	Rajeev	200000
1	Sumit	140000
2	Aviral	90000

```
# Store the data in a dictionary
series_dict = {'First Series' : pd.Series([10, 20, 30, 40]),
               'Second Series' : pd.Series([10, 20, 30, 40])}
```

```
# Create the DataFrame using the dictionary
df = pd.DataFrame(series_dict)
```

```
# Print the dataframe
df
```



	First Series	Second Series
0	10	10
1	20	20
2	30	30
3	40	40

✓ Using a list of dictionaries

```
# Initialise the list of dictionaries
list_of_dicts = [{'a': 1, 'b': 2, 'c': 3}, {'a': 10, 'b': 20, 'c': 30}]

# Create the DataFrame
df = pd.DataFrame(list_of_dicts)
```

```
# Print the dataframe
df
```



	a	b	c
0	1	2	3
1	10	20	30

```
# What happens if the number of keys are different in the two dictionaries
list_of_dicts = [{'a': 2, 'c': 3}, {'a': 10, 'b': 20, 'c': 30}]
df = pd.DataFrame(list_of_dicts, index = ['first', 'second'])
df
```



	a	c	b
first	2	3	NaN
second	10	30	20

✓ There are other ways too!

```
# List of names
actor_names = ['Ryan Reynolds', 'Benedict Cumberbatch', 'Robert Downey Jr.', 'Chris Evans']

# List of their ages
actor_ages = [48, 62, 54, np.nan]

# Get the list of tuples by zipping the two lists together
list_of_tuples = list(zip(actor_names, actor_ages))

# Converting the list of tuples into pandas DataFrame
actor_df = pd.DataFrame(list_of_tuples, columns = ['Name', 'Age'])

# Print the dataframe
actor_df
```



	Name	Age
0	Ryan Reynolds	48.0
1	Benedict Cumberbatch	62.0
2	Robert Downey Jr.	54.0
3	Chris Evans	NaN

✓ So we have seen a variety of ways in which we can create simple dataframes. Now let us explore few methods and attributes associated with the pandas dataframes in Python

> We can print the indices of our dataframe using the `index` attribute

Let us create a dataframe with 100 rows and 5 columns

```
# Create an array of shape (100,5)
our_array = np.random.randn(100, 5)
df = pd.DataFrame(our_array, columns=['A', 'B', 'C', 'D', 'E'])
df
```



	A	B	C	D	E
0	0.485291	0.946751	0.927848	-0.284047	1.043190
1	-0.967732	1.142556	-1.839190	1.024524	0.595620
2	-0.818485	0.649729	-0.172276	-0.005500	1.579862
3	0.969116	-1.168238	1.133358	-0.512334	0.361157
4	-1.603277	-0.464856	-0.301436	-0.156772	-2.563144
...	...	...	...	...	...
95	0.431810	1.045151	-0.720390	0.019411	0.507711
96	0.169092	0.311790	-0.551278	-0.599970	0.142935
97	-0.079926	0.051081	-0.971247	1.813261	0.288375
98	-0.216824	-0.453124	1.208966	0.251313	0.119297
99	1.127710	1.745433	1.488644	-0.429668	1.430431

100 rows x 5 columns

• `head()`

Prints the first n rows of the dataframe. By default it prints the first 5 rows

```
# We can use the head() method
df.head()
```



	A	B	C	D	E
0	0.485291	0.946751	0.927848	-0.284047	1.043190
1	-0.967732	1.142556	-1.839190	1.024524	0.595620
2	-0.818485	0.649729	-0.172276	-0.005500	1.579862
3	0.969116	-1.168238	1.133358	-0.512334	0.361157
4	-1.603277	-0.464856	-0.301436	-0.156772	-2.563144

```
# In fact we can also print the first 10 rows
df.head(10)
```

	A	B	C	D	E
0	0.485291	0.946751	0.927848	-0.284047	1.043190
1	-0.967732	1.142556	-1.839190	1.024524	0.595620
2	-0.818485	0.649729	-0.172276	-0.005500	1.579862
3	0.969116	-1.168238	1.133358	-0.512334	0.361157
4	-1.603277	-0.464856	-0.301436	-0.156772	-2.563144
5	0.446832	-0.328044	-1.655902	-0.571027	-0.596503
6	0.457607	0.117157	-0.209278	1.710380	0.516425
7	-1.323449	-1.769824	-0.270849	-1.484332	-0.883924
8	0.368566	-0.304273	-1.224694	1.067345	-1.184440
9	-1.424237	-0.003178	0.747596	1.330171	-0.491450

`tail()`

Prints the last n rows of the dataframe. By default it prints the last 5 rows

```
df.tail()
```

	A	B	C	D	E
95	0.431810	1.045151	-0.720390	0.019411	0.507711
96	0.169092	0.311790	-0.551278	-0.599970	0.142935
97	-0.079926	0.051081	-0.971247	1.813261	0.288375
98	-0.216824	-0.453124	1.208966	0.251313	0.119297
99	1.127710	1.745433	1.488644	-0.429668	1.430431

```
df.tail(10)
```

	A	B	C	D	E
90	1.787557	-1.478197	-1.016233	-0.249155	-1.940020
91	0.062312	-1.467833	1.559150	-1.305075	-0.809776
92	1.284551	0.215340	0.944348	1.943390	0.464716
93	-1.768649	0.111345	0.806180	0.993246	-0.234683
94	-0.098544	0.087786	-1.262775	0.504647	-0.688150
95	0.431810	1.045151	-0.720390	0.019411	0.507711
96	0.169092	0.311790	-0.551278	-0.599970	0.142935
97	-0.079926	0.051081	-0.971247	1.813261	0.288375
98	-0.216824	-0.453124	1.208966	0.251313	0.119297
99	1.127710	1.745433	1.488644	-0.429668	1.430431

`columns`

Prints the list of columns in a dataframe

```
list(df.columns)
```

```
['A', 'B', 'C', 'D', 'E']
```

```
list(actor_df.columns)
```

```
['Name', 'Age']
```

. ✓ info()

Prints the basic information about our dataframe

actor_df



	Name	Age
0	Ryan Reynolds	48.0
1	Benedict Cumberbatch	62.0
2	Robert Downey Jr.	54.0
3	Chris Evans	NaN

actor_df.info()



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Name    4 non-null       object
1   Age     3 non-null       float64
dtypes: float64(1), object(1)
memory usage: 196.0+ bytes
```

. ✓ rename()

Rename the columns of the dataframe

actor_df



	Name	Age
0	Ryan Reynolds	48.0
1	Benedict Cumberbatch	62.0
2	Robert Downey Jr.	54.0
3	Chris Evans	NaN

```
actor_df.rename(columns = {'Name':'Actor Name','Age' : 'Actor Age'}, inplace=True)
```

Start coding or [generate](#) with AI.

actor_df



	Actor Name	Actor Age
0	Ryan Reynolds	48.0
1	Benedict Cumberbatch	62.0
2	Robert Downey Jr.	54.0
3	Chris Evans	NaN

employee_df



	Employee Name	Income
0	Rajeev	200000
1	Sumit	140000
2	Aviral	900000


```
employee_df.rename(columns = {'Employee Name':'Name'}, inplace=True)
```

```
employee_df
```



	Name	Income
0	Rajeev	200000
1	Sumit	140000
2	Aviral	90000

```
employee_df = employee_df.rename({'Employee Name':'Name'},axis=1) #changes along column
```

```
employee_df
```



	Name	Income
0	Rajeev	200000
1	Sumit	140000
2	Aviral	90000

```
employee_df = employee_df.rename({1:'one'},axis=0) #changes along row
```

```
employee_df
```



	Name	Income
0	Rajeev	200000
one	Sumit	140000
2	Aviral	90000

Up till now, we have been creating our own simple dataframes and exploring them. But of

✓ **course life is not that simple. Most of the data you would deal with in your jobs would be a lot bigger and messier. Let us start with first loading these data in our Python environment.**

✓ Loading a CSV (Comma-Separated Values) file

Start coding or [generate](#) with AI.

```
import pandas as pd
```

```
imdb_df=pd.read_csv('imdb_data.csv')
```

```
imdb_df.head()
```

	id	belongs_to_collection	budget	genres	homepage	imdb_id	original_language	original_title	overview
0	1	[[{'id': 313576, 'name': 'Hot Tub Time Machine ...	14000000	[[{'id': 35, 'name': 'Comedy'}]]	NaN	tt2637294	en	Hot Tub Time Machine 2	When Lou, who has become the "father of the In...
1	2	[[{'id': 107674, 'name': 'The Princess Diaries ...	40000000	[[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'nam...	NaN	tt0368933	en	The Princess Diaries 2: Royal Engagement	Mia Thermopolis is now a college graduate and ...
2	3	NaN	3300000	[[{'id': 18, 'name': 'Drama'}]]	http://sonyclassics.com/whiplash/	tt2582802	en	Whiplash	Under the direction of a ruthless instructor, ...
3	4	NaN	1200000	[[{'id': 53, 'name': 'Thriller'}, {'id': 18, 'n...	http://kahaanithefilm.com/	tt1821480	hi	Kahaani	Vidya Bagchi (Vidya Balan) arrives in Kolkata ...
4	5	NaN	0	[[{'id': 28, 'name': 'Action'}, {'id': 53, 'nam...	NaN	tt1380152	ko	마린보이	Marine Boy is the story of a former national s...

5 rows × 23 columns

imdb_df.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 23 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   id                    3000 non-null  int64
 1   belongs_to_collection 604 non-null   object
 2   budget                3000 non-null  int64
 3   genres                2993 non-null  object
 4   homepage               946 non-null   object
 5   imdb_id               3000 non-null  object
 6   original_language     3000 non-null  object
 7   original_title        3000 non-null  object
 8   overview              2992 non-null  object
 9   popularity            3000 non-null  float64
10   poster_path           2999 non-null  object
11   production_companies  2844 non-null  object
12   production_countries  2945 non-null  object
13   release_date          3000 non-null  object
14   runtime               2998 non-null  float64
15   spoken_languages      2980 non-null  object
16   status                3000 non-null  object
17   tagline               2403 non-null  object
18   title                 3000 non-null  object
19   Keywords              2724 non-null  object
20   cast                  2987 non-null  object
21   crew                  2984 non-null  object
22   revenue               3000 non-null  int64
dtypes: float64(2), int64(3), object(18)
memory usage: 539.2+ KB

```

list(imdb_df.columns)

```

['id',
 'belongs_to_collection',
 'budget',
 'genres',
 'homepage',
 'imdb_id',
 'original_language',
 'original_title',

```

```
'overview',  
'popularity',  
'poster_path',  
'production_companies',  
'production_countries',  
'release_date',  
'runtime',  
'spoken_languages',  
'status',  
'tagline',  
'title',  
'Keywords',  
'cast',  
'crew',  
'revenue']
```

```
imdb_df.shape
```